Northwind SQL Practice Questions

Parts A — with Solutions and Explanations

Part A — Foundational SELECT Queries

The questions in this section focus on core SELECT syntax: choosing columns, filtering rows with WHERE, using comparison and pattern-matching operators, and writing simple aggregate queries. All queries use the Northwind sample database.

Question 1

Task: Write a query to list the CategoryID, CategoryName, and Description of every category in the Northwind database.

Hint: Look at the Categories table. Use SELECT to pick only the columns you need.

Question 2

Task: Write a query to find all products where the quantity per unit contains the word 'box' (e.g., '12 boxes' or '24 - 12 oz boxes').

Hint: Use the LIKE operator with wildcard characters to search for partial text matches inside the QuantityPerUnit column of the Products table.

Question 3

Task: Write a query to list all products that are currently discontinued. Display the ProductID, ProductName, and Discontinued columns.

Hint: The Discontinued column in the Products table stores 1 for discontinued and 0 for active.

Question 4

Task: Write a query to display the full name of every employee by combining their first and last name into a single column. Label the combined column FullName.

Hint: Look at the Employees table. Use string concatenation (+ in SQL Server, || in SQLite/PostgreSQL) to combine FirstName and LastName with a space between them.

Question 5

Task: Write a query to list all customers located in either Germany or France. Display the CustomerID, CompanyName, and Country.

Hint: You can use IN to test against a list of values instead of writing multiple OR conditions.

Question 6

Task: How many total orders have been placed in the Orders table? Write a query that returns this count and labels it TotalOrders.

Hint: Use the COUNT() aggregate function on the primary key of the Orders table.

Question 7

Task: Write a query to list all suppliers whose contact title is either 'Sales Representative' or 'Sales Manager'. Show SupplierID, CompanyName, ContactName, and ContactTitle.

Hint: Use the IN operator or two OR conditions in your WHERE clause against the Suppliers table.

Question 8

Task: Write a query to find all orders that were shipped to the USA. Display OrderID, CustomerID, ShipCountry, and ShippedDate.

Hint: Filter the Orders table using the ShipCountry column.

Question 9

Task: Examine the Orders table. It records a required date and an actual shipped date. Write a query to find all orders where the product was shipped after the required date (i.e., the shipment was late). Display OrderID, CustomerID, RequiredDate, and ShippedDate.

Hint: Compare two date columns using the > operator in a WHERE clause.

Question 10

Task: Examine the Products table. It contains a column called UnitsInStock and a column called ReorderLevel. Write a query that identifies products that need to be reordered — that is, products where the current stock is at or below the reorder level and the product is NOT discontinued.

Hint: Combine two conditions in the WHERE clause using AND.

Part A Solutions

Question 1

```sql
USE northwind;

-- Part A Question 1
SELECT CategoryID, CategoryName, Description
FROM Categories;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cel

CategoryID	CategoryName	Description
1	Beverages	Soft drinks, coffees, teas, beers, and ales
2	Condiments	Sweet and savory sauces, relishes, spreads, an...
3	Confections	Desserts, candies, and sweet breads
4	Dairy Products	Cheeses
5	Grains/Cereals	Breads, crackers, pasta, and cereal
6	Meat/Poultry	Prepared meats
7	Produce	Dried fruit and bean curd
8	Seafood	Seaweed and fish
NULL	NULL	NULL

Categories 1

Output

Action Output

#	Time	Action	Message
1	09:56:49	USE northwind	0 row(s) affected
2	09:56:49	SELECT CategoryID, CategoryName, Description FROM Categories LIMI...	8 row(s) returned

Explanation

- SELECT specifies the columns to display: CategoryID, CategoryName, and Description.
- FROM Categories tells SQL which table to retrieve the data from.
- No WHERE clause is needed because we want every row in the table.
- This is a foundational SELECT statement — columns are comma-separated and the statement ends with a semicolon.

Question 2

```sql
-- Part A Question 2
SELECT ProductID, ProductName, QuantityPerUnit
FROM Products
WHERE QuantityPerUnit LIKE '%box%';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ProductID	ProductName	QuantityPerUnit
1	Chai	10 boxes x 20 bags
5	Chef Anton's Gumbo Mix	36 boxes
13	Konbu	2 kg box
16	Pavlova	32 - 500 g boxes
19	Teatime Chocolate Biscuits	10 boxes x 12 pieces
20	Sir Rodney's Marmalade	30 gift boxes
47	Zaanse koeken	10 - 4 oz boxes
52	Filo Mix	16 - 2 kg boxes
55	Pt chinois	24 boxes x 2 pies
68	Scottish Longbreads	10 boxes x 8 pieces
77	Original Frankfurter grne ...	12 boxes
NULL	NULL	NULL

Explanation

- LIKE is used for pattern matching in text columns.
- The % wildcard matches any sequence of characters (including none) on either side of the word 'box'.
- Using '%box%' finds values containing 'box' anywhere in the string, whether it appears at the start, middle, or end.
- This is case-insensitive in most SQL dialects (SQL Server, MySQL).

Question 3

```sql
-- Part A Question 3
SELECT ProductID, ProductName, Discontinued
FROM Products
WHERE Discontinued = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

ProductID	ProductName	Discontinued
5	Chef Anton's Gumbo Mix	1
9	Mishi Kobe Niku	1
17	Alice Mutton	1
24	Guaran Fantstica	1
28	Rssle Sauerkraut	1
29	Thringer Rostbratwurst	1
42	Singaporean Hokkien Fried Mee	1
53	Perth Pasties	1
NULL	NULL	NULL

Explanation

- In the Northwind database the Discontinued column is a bit/boolean field.
- A value of 1 means the product has been discontinued; 0 means it is still active.
- The WHERE clause filters rows to show only discontinued products.
- This demonstrates filtering on a boolean/bit column — a very common pattern in database applications.

Question 4:

```sql
-- Part A Question 4
SELECT EmployeeID,
       FirstName + ' ' + LastName AS FullName,
       Title
FROM Employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

EmployeeID	FullName	Title
1	0	Sales Representative
2	0	Vice President, Sales
3	0	Sales Representative
4	0	Sales Representative
5	0	Sales Manager
6	0	Sales Representative
7	0	Sales Representative
8	0	Inside Sales Coordinator
9	0	Sales Representative

Result 4

Output

Action Output

#	Time	Action	Message
1	10:09:22	SELECT EmployeeID, FirstName + ' ' + LastName AS FullName, ...	9 row(s) returned

Explanation

- The + operator in SQL Server (or || in other dialects) joins strings together.
- ' ' (a single space between two single quotes) adds a space between the first and last name.
- AS FullName gives the combined expression a readable column alias.
- Aliases are applied with the AS keyword and appear as the column header in the result set.

Question 5

```sql
-- Part A Question 5
SELECT CustomerID, CompanyName, Country
FROM Customers
WHERE Country IN ('Germany', 'France');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conte

CustomerID	CompanyName	Country
ALFKI	Alfreds Futterkiste	Germany
BLAUS	Blauer See Delikatessen	Germany
BLONP	Blondesddsl pre et fils	France
BONAP	Bon app'	France
DRACD	Drachenblut Delikatessen	Germany
DUMON	Du monde entier	France
FOLIG	Folies gourmandes	France
FRANK	Frankenversand	Germany
FRANR	France restauration	France
KOENE	Kniglich Essen	Germany
LACOR	La corne d'abondance	France
LAMAI	La maison d'Asie	France
LEHMS	Lehmanns Marktstand	Germany

Customers 6

Output

Action Output

#	Time	Action	Message
1	10:12:29	SELECT CustomerID, CompanyName, Country FROM Customers WHER...	22 row(s) returned

Explanation

- The IN operator is a shorthand for multiple OR equality checks.
- WHERE Country IN ('Germany', 'France') is equivalent to WHERE Country = 'Germany' OR Country = 'France'.
- String values must be enclosed in single quotes.
- IN keeps the query readable when checking against several possible values.

Question 6

```
32 •  SELECT COUNT(OrderID) AS TotalOrders
33    FROM Orders;
34
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TotalOrders
830

Result 7

Output

Action Output

#	Time	Action	Message
1	10:15:50	SELECT COUNT(OrderID) AS TotalOrders FROM Orders LIMIT 0, 1000	1 row(s) returned

Explanation

- COUNT() is an aggregate function that counts the number of non-NULL values in a column.
- Counting on the primary key (OrderID) guarantees that every row is counted.
- AS TotalOrders assigns a meaningful alias to the result column.
- Aggregate functions collapse multiple rows into a single summary value — here, one row with the total.

Question 7

```
-- Part A Question 7
SELECT SupplierID, CompanyName, ContactName, ContactTitle
FROM Suppliers
WHERE ContactTitle IN ('Sales Representative', 'Sales Manager');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

SupplierID	CompanyName	ContactName	ContactTitle
3	Grandma Kelly's Homestead	Regina Murphy	Sales Representative
8	Specialty Biscuits, Ltd.	Peter Wilson	Sales Representative
11	Heli Swaren GmbH & Co. KG	Petra Winkler	Sales Manager
14	Formaggi Fortini s.r.l.	Elio Rossi	Sales Representative
17	Svensk Sjfda AB	Michael Bjrn	Sales Representative
18	Aux joyeux ecclsiastiques	Guylne Nodier	Sales Manager

Suppliers 8 × Apply

Output

Action Output

#	Time	Action	Message
1	10:19:37	SELECT SupplierID, CompanyName, ContactName, ContactTitle FROM ...	10 row(s) returned

Explanation

- This query targets the Suppliers table using IN for a clean multi-value filter.
- The IN list contains string literals, each enclosed in single quotes and separated by commas.
- This approach is easy to extend — just add more values inside the parentheses.
- Compared to multiple OR conditions, IN is typically more readable and equally efficient.

Question 8

```sql
-- Part A Question 8
SELECT OrderID, CustomerID, ShipCountry, ShippedDate
FROM Orders
WHERE ShipCountry = 'USA';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Co

OrderID	CustomerID	ShipCountry	ShippedDate
10262	RATTC	USA	1996-07-25 00:00:00
10269	WHITC	USA	1996-08-09 00:00:00
10271	SPLIR	USA	1996-08-30 00:00:00
10272	RATTC	USA	1996-08-06 00:00:00
10294	RATTC	USA	1996-09-05 00:00:00
10305	OLDWO	USA	1996-10-09 00:00:00

Orders 9

Output

Action Output

#	Time	Action	Message
1	10:24:18	SELECT OrderID, CustomerID, ShipCountry, ShippedDate FROM Orders ...	122 row(s) returned

Explanation

- This is a straightforward equality filter on a text column.
- String comparisons in SQL are enclosed in single quotes.
- The ShippedDate column will show NULL for orders not yet shipped — a real-world consideration.
- Column selection (not SELECT *) is a good practice: it makes the query's intent clear and improves performance on large tables.

Question 9

```sql
-- Part A Question 9
SELECT OrderID, CustomerID, RequiredDate, ShippedDate
FROM Orders
WHERE ShippedDate > RequiredDate;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell C

OrderID	CustomerID	RequiredDate	ShippedDate
10264	FOLKO	1996-08-21 00:00:00	1996-08-23 00:00:00
10271	SPLIR	1996-08-29 00:00:00	1996-08-30 00:00:00
10280	BERGS	1996-09-11 00:00:00	1996-09-12 00:00:00
10302	SUPRD	1996-10-08 00:00:00	1996-10-09 00:00:00
10309	HUNGO	1996-10-17 00:00:00	1996-10-23 00:00:00
10320	WARTH	1996-10-17 00:00:00	1996-10-18 00:00:00

Orders 10

Output

Action Output

#	Time	Action	Message
1	10:27:49	SELECT OrderID, CustomerID, RequiredDate, ShippedDate FROM Order...	37 row(s) returned

Explanation

• You can compare two columns directly in a WHERE clause — you are not limited to comparing a column against a fixed value.

• ShippedDate > RequiredDate returns only rows where the shipped date is later (greater) than the required date, meaning the order was late.

• Rows where ShippedDate IS NULL (order not yet shipped) are automatically excluded because comparisons with NULL always evaluate to unknown/false in SQL.

Question 10

```sql
-- Part A Question 10
SELECT ProductID, ProductName, UnitsInStock, ReorderLevel, Discontinued
FROM Products
WHERE UnitsInStock <= ReorderLevel
  AND Discontinued = 0;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

ProductID	ProductName	UnitsInStock	ReorderLevel	Discontinued
2	Chang	17	25	0
3	Aniseed Syrup	13	25	0
11	Queso Cabrales	22	30	0
21	Sir Rodney's Scones	3	5	0
30	Nord-Ost Matjeshering	10	15	0
31	Gorgonzola Telino	0	20	0

Products 11 × Apply

Output

Action Output

#	Time	Action	Message
1	10:30:29	SELECT ProductID, ProductName, UnitsInStock, ReorderLevel, Disconti...	18 row(s) returned

Explanation

• <= means 'less than or equal to', so the filter catches products at or below their reorder threshold.

• AND requires BOTH conditions to be true for a row to appear in results.

• Adding AND Discontinued = 0 avoids flagging products that are no longer sold — a practical real-world requirement.

• This query demonstrates combining numeric comparisons with boolean/bit column filters.

Part B — Sorting, Grouping, and Joining

The questions in this section build on Part A by introducing ORDER BY for sorting, GROUP BY and aggregate functions (COUNT, SUM, AVG) for summarisation, HAVING for filtering groups, and JOIN for combining data from multiple tables.

Question 1

Task: Write a query to list ProductID, ProductName, UnitPrice, and UnitsInStock from the Products table. Order the results by UnitPrice from highest to lowest.

Hint: Use the ORDER BY clause with DESC to sort in descending order.

SQL Solution

```
SELECT ProductID, ProductName, UnitPrice, UnitsInStock
FROM Products
ORDER BY UnitPrice DESC;
```

Explanation

• ORDER BY controls the sequence of rows in the result set.

• DESC (descending) sorts from the largest value to the smallest — here, most expensive products appear first.

• ASC (ascending) is the default and does not need to be written explicitly.

• ORDER BY is placed after the WHERE clause (if any) and is the last clause in a basic SELECT statement.

Question 2

Task: Write a query to list all customers. Order them first by Country (A–Z), then by CompanyName (A–Z) within each country.

Hint: You can list multiple columns in the ORDER BY clause separated by commas.

SQL Solution

```
SELECT CustomerID, CompanyName, ContactName, Country
FROM Customers
ORDER BY Country ASC, CompanyName ASC;
```

Explanation

• Multiple columns in ORDER BY create a multi-level sort.

• The first column listed (Country) is the primary sort key.

• The second column (CompanyName) is the secondary sort key — it only comes into play when two or more rows share the same Country value.

• ASC is the default direction but is written here for clarity.

Question 3

Task: Write a query that counts the total number of products in each category. Display the CategoryID and the count, labeling the count ProductCount. Order the results so the category with the most products appears first.

Hint: Use COUNT() with GROUP BY on CategoryID, then ORDER BY the count descending.

SQL Solution

```
SELECT CategoryID, COUNT(ProductID) AS ProductCount
FROM Products
GROUP BY CategoryID
ORDER BY ProductCount DESC;
```

Explanation

• GROUP BY CategoryID splits the products into groups, one per category.

• COUNT(ProductID) then counts how many products exist within each group.

• ORDER BY ProductCount DESC sorts by the alias, so the largest group appears first.

• This is a classic aggregation pattern: SELECT a grouping column + aggregate, GROUP BY that column, then ORDER BY the result.

Question 4

Task: Write a query against the Order Details table that calculates the total revenue per order. Total revenue for a line is UnitPrice × Quantity × (1 − Discount). Label the total Revenue and display it alongside OrderID. Order the results by Revenue from largest to smallest.

Hint: Use SUM() on the calculated expression and GROUP BY OrderID.

SQL Solution

```
SELECT OrderID,
    SUM(UnitPrice * Quantity * (1 - Discount)) AS Revenue
FROM [Order Details]
GROUP BY OrderID
ORDER BY Revenue DESC;
```

Explanation

• SUM() is an aggregate that totals values across all rows in each group.

• The expression UnitPrice * Quantity * (1 - Discount) computes the discounted line total for each row.

• GROUP BY OrderID groups all line items belonging to the same order so SUM adds them together.

• The table name is [Order Details] — square brackets are required in SQL Server when a table name contains a space.

Question 5

Task: Write a query to list the number of orders placed by each employee. Show EmployeeID and label the count OrderCount. Only include employees who have handled more than 50 orders. Order by OrderCount, largest first.

Hint: Use GROUP BY EmployeeID, then filter groups with HAVING instead of WHERE.

SQL Solution

```
SELECT EmployeeID, COUNT(OrderID) AS OrderCount
FROM Orders
GROUP BY EmployeeID
HAVING COUNT(OrderID) > 50
ORDER BY OrderCount DESC;
```

Explanation

• HAVING is to GROUP BY what WHERE is to individual rows — it filters after aggregation.

• You cannot use WHERE to filter by an aggregate (e.g., WHERE COUNT(...) > 50) because WHERE runs before grouping.

• HAVING COUNT(OrderID) > 50 keeps only groups (employees) where the order count exceeds 50.

• Always remember: WHERE → GROUP BY → HAVING → ORDER BY is the logical execution order.

Question 6

Task: Write a query that lists each shipper (ShipVia) and counts the number of orders assigned to them. Label the count OrderCount. Order results by ShipperID ascending.

Hint: Use GROUP BY on the ShipVia column in the Orders table.

SQL Solution

```
SELECT ShipVia AS ShipperID,
    COUNT(OrderID) AS OrderCount
FROM Orders
GROUP BY ShipVia
ORDER BY ShipVia ASC;
```

Explanation

• GROUP BY ShipVia groups all orders by the shipper used.
• COUNT(OrderID) counts the orders in each group.
• AS ShipperID gives the grouped column a more descriptive alias.
• This pattern is useful for quickly discovering the distribution of orders across shipping carriers.

Question 7

Task: List every product together with the name of its category. Display ProductID, ProductName, and CategoryName. Order alphabetically by CategoryName, then by ProductName within each category.

Hint: Join the Products table to the Categories table on the CategoryID column.

SQL Solution

```
SELECT p.ProductID, p.ProductName, c.CategoryName
FROM Products AS p
JOIN Categories AS c ON p.CategoryID = c.CategoryID
```

```
ORDER BY c.CategoryName ASC, p.ProductName ASC;
```

Explanation

• JOIN (INNER JOIN) combines rows from two tables where the join condition is met.

• ON p.CategoryID = c.CategoryID matches each product to its category using the shared key.

• Table aliases (p, c) make the query shorter and clarify which table each column comes from — especially important when both tables share column names.

• The ORDER BY clause uses two sort keys: primary by CategoryName, secondary by ProductName.

Question 8

Task: Write a query that shows each order and the company name of the customer who placed it. Display OrderID, OrderDate, and CompanyName. Order by OrderDate from most recent to oldest.

Hint: Join the Orders table to the Customers table on CustomerID.

SQL Solution

```
SELECT o.OrderID, o.OrderDate, c.CompanyName
FROM Orders AS o
JOIN Customers AS c ON o.CustomerID = c.CustomerID
ORDER BY o.OrderDate DESC;
```

Explanation

• The JOIN links orders to customers through the shared CustomerID foreign key.

• Without the JOIN you could only see the CustomerID code in Orders; with it you see the human-readable company name from Customers.

• ORDER BY o.OrderDate DESC shows the newest orders at the top — a common reporting requirement.

• Qualifying column names with table aliases (o.OrderID, c.CompanyName) prevents ambiguity.

Question 9

Task: Write a query that calculates the average unit price of products in each category. Display CategoryName and the average price labeled AvgPrice, rounded to 2 decimal places. Only show categories where the average price is greater than $20. Order by AvgPrice descending.

Hint: Join Products and Categories, use AVG() with GROUP BY CategoryID, filter with HAVING, and use ROUND().

SQL Solution

```
SELECT c.CategoryName,
    ROUND(AVG(p.UnitPrice), 2) AS AvgPrice
FROM Products AS p
JOIN Categories AS c ON p.CategoryID = c.CategoryID
GROUP BY c.CategoryID, c.CategoryName
HAVING AVG(p.UnitPrice) > 20
ORDER BY AvgPrice DESC;
```

Explanation

• AVG() calculates the mean of UnitPrice within each category group.

• ROUND(..., 2) trims the result to 2 decimal places for a clean currency display.

• GROUP BY must include c.CategoryName (in addition to c.CategoryID) because it appears in the SELECT — a requirement of standard SQL.

• HAVING AVG(p.UnitPrice) > 20 filters out categories with a lower average price — note the aggregate is repeated in HAVING, not the alias.

Question 10

Task: Write a query that lists each employee's full name along with the total number of orders they have processed. Show FullName (combined FirstName + LastName) and OrderCount. Only include employees who have processed at least 70 orders. Order by OrderCount descending, then by FullName ascending.

Hint: Join Employees and Orders on EmployeeID, aggregate with COUNT and GROUP BY, then filter with HAVING.

SQL Solution

```
SELECT e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS FullName,
```

```
    COUNT(o.OrderID) AS OrderCount
FROM Employees AS e
JOIN Orders AS o ON e.EmployeeID = o.EmployeeID
GROUP BY e.EmployeeID, e.FirstName, e.LastName
HAVING COUNT(o.OrderID) >= 70
ORDER BY OrderCount DESC, FullName ASC;
```

Explanation

- The JOIN links each order to the employee who handled it via EmployeeID.
- String concatenation (FirstName + ' ' + LastName) builds the full name directly in the SELECT.
- GROUP BY must list all non-aggregated columns in SELECT: EmployeeID, FirstName, and LastName.
- HAVING COUNT(...) >= 70 keeps only high-volume employees.
- The two-column ORDER BY demonstrates how to resolve ties — here, alphabetically by full name when two employees have the same order count.